

SPI Driver Analysis

Introduction

The SPI driver provides communication between the STM32F446xx microcontroller and external SPI devices. It hides register-level programming and offers simple APIs for initialization, transmission, reception, and interrupt handling.

How the SPI Driver Works

The driver first enables the peripheral clock and configures the CR1 register with device mode, bus configuration, clock speed, data frame format, CPOL, CPHA and software slave management settings. After initialization the SPI peripheral is enabled for communication.

Data Transmission

SPI_SendData() waits for the TXE flag and writes data into the data register. It supports both 8-bit and 16-bit transfers and works as a blocking function.

Data Reception

SPI_ReceiveData() waits for the RXNE flag and reads incoming data from the data register into the receive buffer.

Interrupt Mode

SPI_SendDataIT() and SPI_ReceiveDataIT() perform non-blocking communication by storing buffer information and enabling TXE or RXNE interrupts.

Interrupt Handling

SPI_IRQHandling() checks TXE, RXNE and OVR flags and calls helper functions to process transmission, reception and overrun errors.

Peripheral Control APIs

Functions such as SPI_PeripheralControl(), SPI_SSICongfig(), SPI_SSOEConfig(), SPI_GetFlagStatus() and SPI_ClearOVRflag() help control SPI operation.

Conclusion

The SPI driver makes SPI communication simple and reusable. It hides hardware complexity and provides organized APIs for embedded applications.

Main APIs

API	Purpose
SPI_PeriClockControl()	Enable/Disable SPI clock
SPI_Init()	Initialize SPI
SPI_SendData()	Blocking transmit
SPI_ReceiveData()	Blocking receive

SPI_SendDataIT()	Interrupt transmit
SPI_ReceiveDataIT()	Interrupt receive
SPI_IRQHandling()	Handle SPI interrupts
SPI_PeripheralControl()	Enable/Disable SPI
SPI_GetFlagStatus()	Check status flags
SPI_ClearOVRFlag()	Clear overrun flag